

BPSK_receiver

```
1  import numpy as np
2
3  from Converter_dB import convert_SNR_dB_to_SNR, convert_SNR_to_SNR_dB
4
5
6  PI = np.pi
7
8  def int_to_bits_lsb(value: int, length: int) -> np.ndarray:
9
10     if value < 0:
11         raise ValueError("value >= 0 must be")
12     if value >= (1 << length):
13         raise ValueError("value size not combinian into import length")
14
15     return np.array([(value >> i) & 1 for i in range(length)],
16                     dtype=np.uint8)
17
18
19
20 class BPSK_receiver:
21
22     def __init__(self, bit_rate, avg_signal_energy, prob_m_arr):
23
24         self._bit_rate          = bit_rate
25         self._T_symb             = 1/self._bit_rate
26         self._E_symb_avg         = avg_signal_energy
27         self._mod_symbols        = self._get_mod_arr()
28         self._demod_symbols      = self._get_demod_dict()
29         self._prob_m             = prob_m_arr
30
31
32
33     def _get_mod_arr(self):
34         arr_signals = np.zeros(2, dtype=complex)
35
36         for numb in range(0, 2):
37             bits = int_to_bits_lsb(numb, 1)
38             arr_signals[numb] = (1/np.sqrt(2))*((1 - 2*bits[0]) + 1j*(1 - 2*bits[0]))
39
40         return arr_signals * np.sqrt(self._E_symb_avg)
41
42
43     def _get_demod_dict(self):
44         dict_simbols = {}
45
46         for num in range(0, 2):
47             dict_simbols[self._mod_symbols[num]] = int_to_bits_lsb(num, 1)
48
```

```

49         return dict_symbols
50
51
52     def _demodulate_signal(self, t, symb_seq, snr_db):
53         # by distance metric, MAP algorithm
54
55         if (np.size(t) - np.size(symb_seq) != 1): raise ValueError("not corre
length in input params, t must be greater than symb_seq by 1")
56
57         snr_lin = convert_SNR_dB_to_SNR(snr_db)
58         N_0 = self._E_symb_avg/snr_lin
59
60         dist_m = np.zeros(np.size(self._mod_symbols))
61
62         for m in range(0, np.size(self._mod_symbols)):
63             signal_m = self._mod_symbols[m]
64             cur_dist_m = 0
65
66             for i in range(0, (np.size(t) - 1)):
67                 cur_dist_m += np.power(np.abs(symb_seq[i] - signal_m), 2) *
(t[i+1]-t[i])
68
69             dist_m[m] = N_0 * np.log(self._prob_m[m]) - cur_dist_m
70
71         return self._demod_symbols[self._mod_symbols[np.argmax(dist_m)]]
72
73
74     def demodulate_symbols_in_time(self, t, signals_in, snr_db):
75
76         if not(np.size(t) >= 2): raise ValueError("t size must be >= 2")
77         if not(np.all(np.diff(t) >= 0)): raise ValueError("t[i] >= t[i+1]")
78         if np.size(t) != np.size(signals_in): raise ValueError("not same leng
th t and symbols_in")
79
80         demod_seq = []
81
82         cur_num_t_start = 0
83
84         for i in range(1, np.size(t)):
85
86             if (t[i] - t[cur_num_t_start] >= self._T_symb):
87                 cur_bit_seq = self._demodulate_signal(t[cur_num_t_start:(i+
1)], signals_in[cur_num_t_start:i], snr_db)
88                 cur_num_t_start = i
89                 demod_seq.append(cur_bit_seq)
90
91             elif (t[i] == t[-1]) and (cur_num_t_start != i):
92                 cur_bit_seq = self._demodulate_signal(t[cur_num_t_start:(i+
1)], signals_in[cur_num_t_start:i], snr_db)
93                 demod_seq.append(cur_bit_seq)
94
95         else:

```

```
96         continue
97
98     return np.concatenate(demod_seq)
```